Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015118**	**Course Name:**	**Artificial Intelligence & Soft Computing Lab**

Name of Student:	**Shreyas sawant**
Roll No. :	**78**
Experiment No.:	**3**
Title of the Experiment:	**Implement Uninformed Search Algorithms (BFS and DFS)**
Date of Performance:	**3/8/2026**
Date of Submission:	**10/8/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty :

Signature :

Date :

2015118: Artificial Intelligence & Soft Computing Lab

Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To implement uninformed search algorithms (Breadth-First Search and Depth-First Search) to reach a goal state.

Objectives:

After completing this experiment, the student will be able to:

1. To understand the concept of uninformed search techniques in Artificial Intelligence. 2. To study the working principles of Breadth-First Search (BFS) and Depth-First Search (DFS).
3. To implement BFS and DFS algorithms for state-space search problems.
4. To compare the traversal strategy of BFS and DFS.
5. To identify the path from the initial state to the goal state using search algorithms.

Tools Required:

- **Hardware:**
  - Desktop/Laptop
- **Software:**
  - Python 3.x
  - Visual Studio Code / PyCharm / Jupyter Notebook
  - Windows/Linux Operating System

Theory:

Search algorithms are fundamental techniques in Artificial Intelligence used to find a solution by exploring the possible states of a problem. Uninformed search algorithms, also known as blind search algorithms, do not possess any domain-specific knowledge about the location of the goal state. They explore the search space systematically using only the information available about the problem.

Breadth-First Search (BFS) explores the search tree level by level, expanding all nodes at the current depth before moving to the next level. It guarantees the shortest path in an unweighted graph but requires more memory. Depth-First Search (DFS) explores one branch of the search tree as deeply as possible before backtracking to explore alternative paths. DFS requires less memory but does not always guarantee the shortest path. These algorithms are widely used in pathfinding, graph traversal, puzzle solving and AI problem-solving applications.

Procedure:

1. Create a graph or tree representing the search space.
2. Define the initial node and the goal node.
3. Implement the Breadth-First Search (BFS) algorithm.
4. Execute the BFS algorithm and record the traversal path.
5. Implement the Depth-First Search (DFS) algorithm.

6. Execute the DFS algorithm and record the traversal path.
7. Compare the outputs obtained from both algorithms and verify whether the goal state is reached.



Description of the Experiment:

This experiment demonstrates the implementation of Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms to traverse a graph or state-space and locate the specified goal state. The algorithms systematically explore the search space using different traversal strategies without employing heuristic information.

The experiment helps in understanding the behaviour, traversal order, and performance of BFS and DFS while solving simple AI search problems.

Detailed Description:

Uninformed search algorithms explore the search space without using any heuristic or additional domain knowledge. In this experiment, a graph or tree structure is considered, where each node represents a state and the edges represent possible transitions between states. The objective is to identify a path from the initial state to the specified goal state.

The Breadth-First Search (BFS) algorithm expands nodes level by level using a queue, ensuring that the shallowest nodes are explored first. In contrast, the Depth-First Search (DFS) algorithm explores one branch completely before backtracking, using a stack or recursion. The traversal sequence and the path obtained from both algorithms are observed and compared to understand their search strategies and suitability for different AI problem-solving scenarios.

Conclusion:

Breadth-First Search and Depth-First Search algorithms were successfully implemented to traverse the search space and reach the specified goal state. The experiment demonstrated the different traversal strategies and applications of uninformed search techniques in Artificial Intelligence..

Implementation:

```
from collections import deque
# Graph
graph = {
 'A': ['B', 'C'],
 'B': ['D', 'E'],
 'C': ['F'],
 'D': [],
 'E': [],
 'F': ['G'],
```

```
 'G': []
}
# Breadth-First Search (BFS)
def bfs(start, goal):
 queue = deque([start])
 visited = set()
```



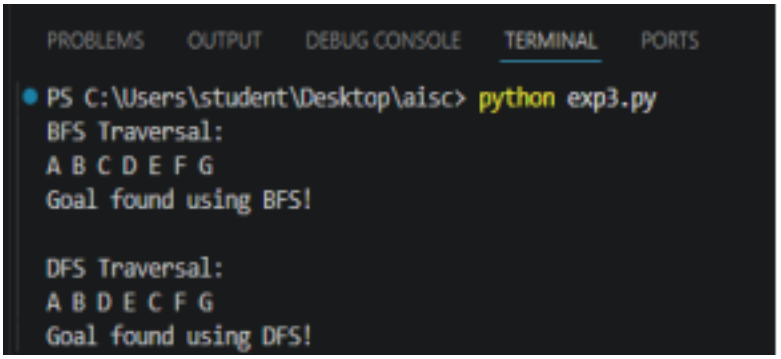


```
 while queue:
 node = queue.popleft()

 if node in visited:
 continue
 print(node, end=" ")
 visited.add(node)
 if node == goal:
 print("\nGoal found using BFS!")
 return
 for neighbour in graph[node]:
 if neighbour not in visited:
 queue.append(neighbour)
 print("\nGoal not found using BFS.")
# Depth-First Search (DFS)
def dfs(start, goal):
 stack = [start]
 visited = set()
 while stack:
 node = stack.pop()
 if node in visited:
 continue
 print(node, end=" ")
 visited.add(node)
 if node == goal:
 print("\nGoal found using DFS!")
 return
 # Reverse order to maintain left-to-right traversal
 for neighbour in reversed(graph[node]):
 if neighbour not in visited:
 stack.append(neighbour)
 print("\nGoal not found using DFS.")
# Initial and goal states
start = 'A'
goal = 'G'
print("BFS Traversal:")
bfs(start, goal)

print("\nDFS Traversal:")
dfs(start, goal)
```

Output:



Questions to be Answered by Students:

1. Which search algorithm explores the search space level by level?

Ans: Breadth-First Search (BFS) explores the search space level by level

2. Which search algorithm was used to reach the goal state in the experiment?

Ans: Both BFS and DFS were used to reach the goal state from the initial state.

3. What traversal order was obtained using BFS and DFS?

Ans: For the given graph:

BFS: A → B → C → D → E → F → G
DFS: A → B → D → E → C → F → G

4. State two observations obtained by comparing the performance of BFS and DFS.

Ans:

- BFS searches level by level and can find the shortest path in an unweighted graph, but it requires more memory.
- DFS searches deeply along one branch and uses less memory, but it does not guarantee the shortest path.

2015118: Artificial Intelligence & Soft Computing Lab